

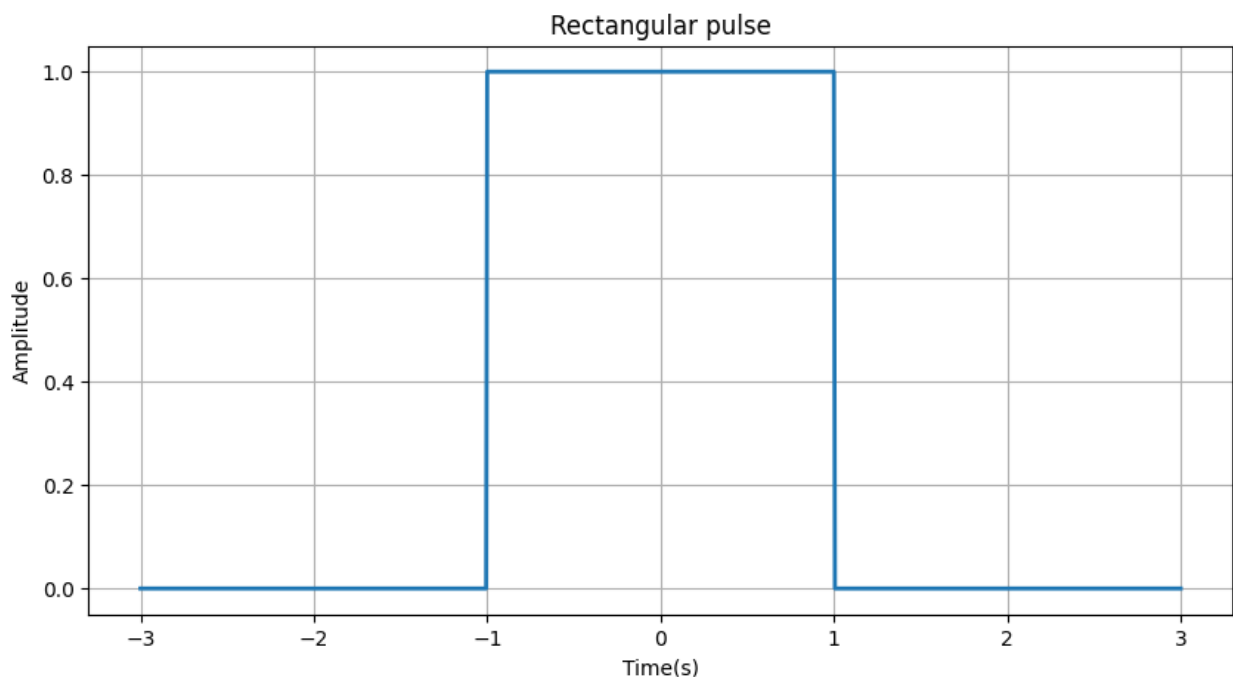
Module 4

Hints and Solutions

Section A — Basic Problems

Problem 1

```
import numpy as np
import matplotlib.pyplot as plt
t = np.linspace(-3,3,1000) #time vector
x = np.where(np.abs(t) <= 1,1,0) #rectangular pulse
plt.figure(figsize=(10,5))
plt.plot(t, x, linewidth=2)
plt.title("Rectangular pulse")
plt.xlabel("Time(s)")
plt.ylabel("Amplitude")
plt.grid(True)
plt.show()
energy = np.trapezoid(x**2, t) #energy
print("Amplitude = ", np.max(x))
print("Width = ", 2)
print("Energy = ", energy)
```



```
Amplitude = 1  
Width = 2  
Energy = 2.0060060060060056
```

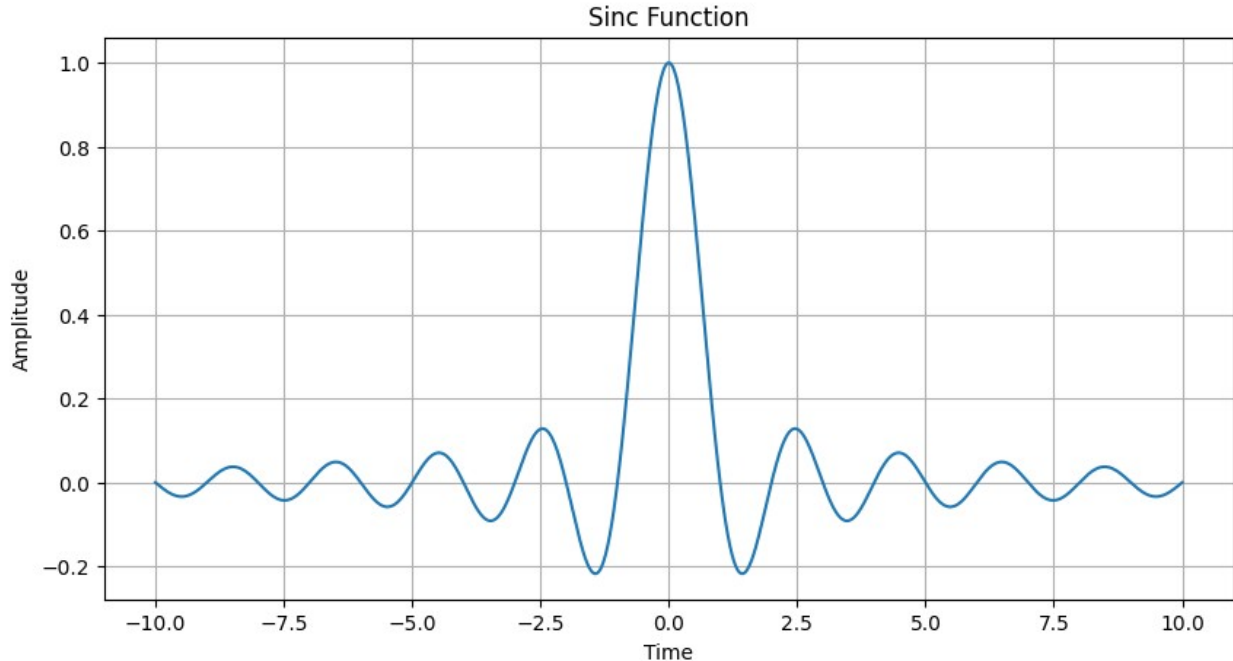
Problem 2

Hint

Sinc Function

Use NumPy's normalized sinc function and observe the central peak and side lobes and identify whether the function is even or odd.

```
import numpy as np  
import matplotlib.pyplot as plt  
t = np.linspace(-10, 10, 1000)  
x = np.sinc(t)  
plt.figure(figsize=(10,5))  
plt.plot(t, x)  
plt.title("Sinc Function")  
plt.xlabel("Time")  
plt.ylabel("Amplitude")  
plt.grid(True)  
plt.show()
```

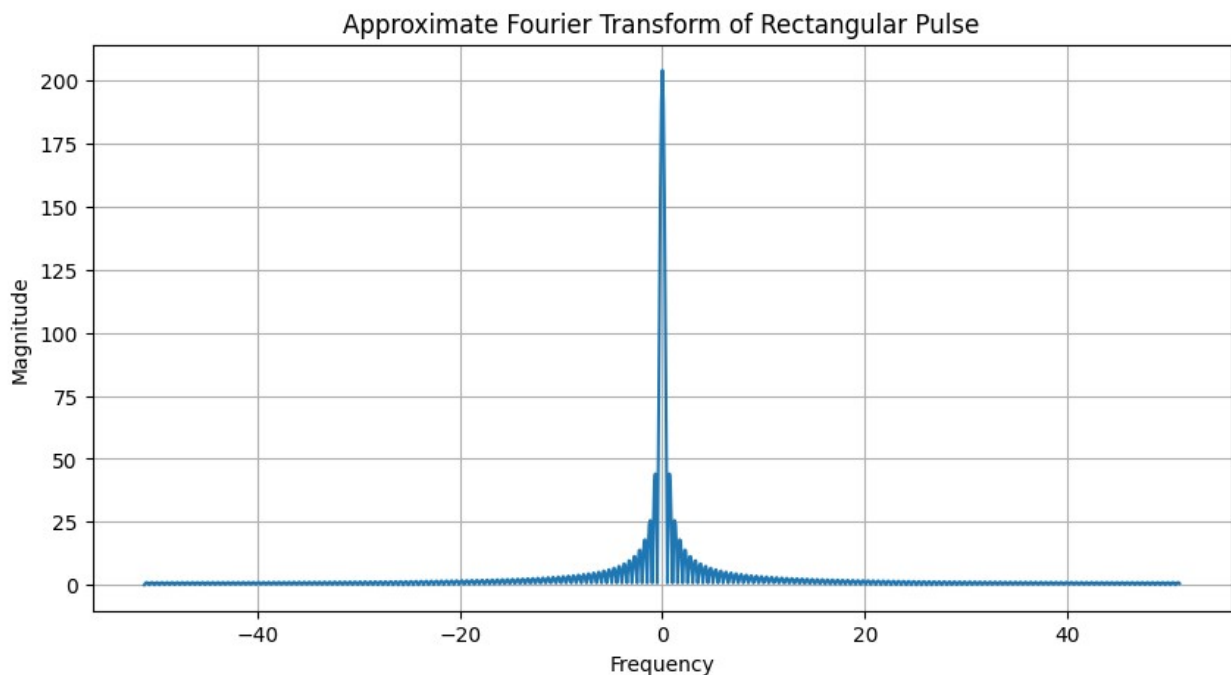


Problem 3

Hint

Generate a rectangular pulse. Use the Fast Fourier Transform (FFT) to approximate the Continuous-Time Fourier Transform. Shift the zero-frequency component to the center.

```
import numpy as np
import matplotlib.pyplot as plt
t = np.linspace(-5,5,1024)
dt = t[1]-t[0]
x = np.where(np.abs(t)<=1,1,0)
X = np.fft.fftshift(np.fft.fft(x))
f = np.fft.fftshift(np.fft.fftfreq(len(t),dt))
plt.figure(figsize=(10,5))
plt.plot(f,np.abs(X))
plt.title("Approximate Fourier Transform of Rectangular Pulse")
plt.xlabel("Frequency")
plt.ylabel("Magnitude")
plt.grid(True)
plt.show()
```



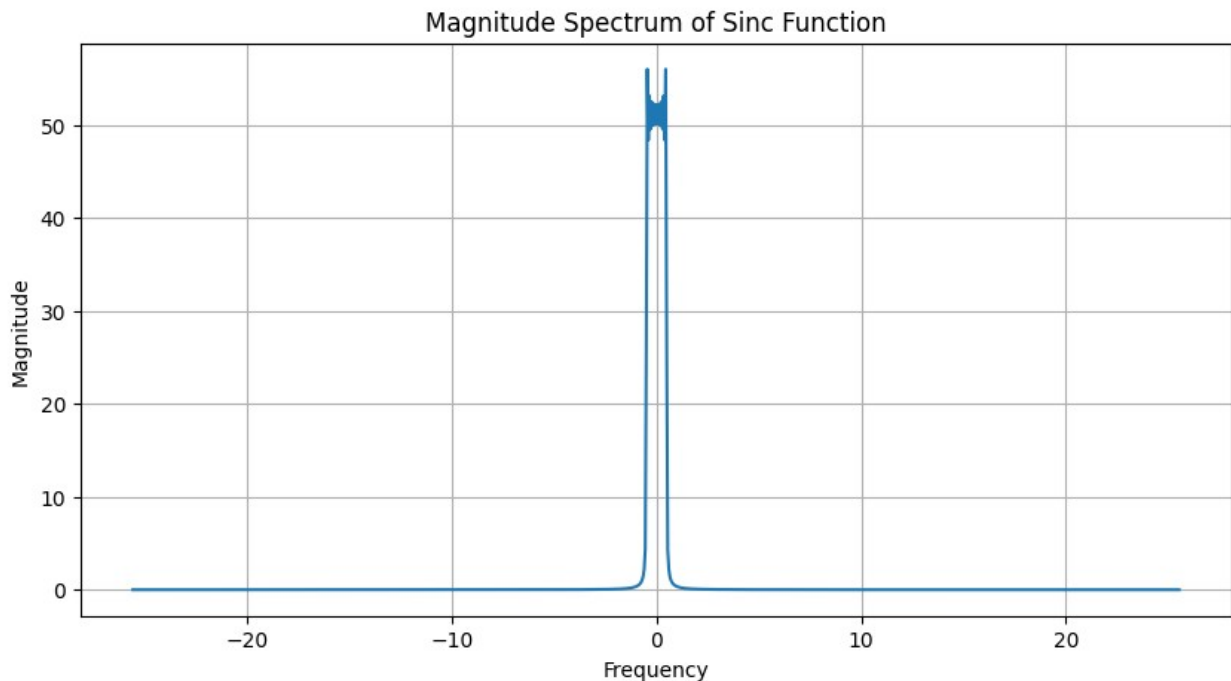
Problem 4

```
import numpy as np
import matplotlib.pyplot as plt
t = np.linspace(-10,10,1024)
dt = t[1]-t[0]
```

```

x = np.sinc(t)
X = np.fft.fftshift(np.fft.fft(x))
f = np.fft.fftshift(np.fft.fftfreq(len(t),dt))
plt.figure(figsize=(10,5))
plt.plot(f,np.abs(X))
plt.title("Magnitude Spectrum of Sinc Function")
plt.xlabel("Frequency")
plt.ylabel("Magnitude")
plt.grid(True)
plt.show()

```



Problem 5

Hint

Create two rectangular pulses with different widths. Compute the FFT of both signals. Compare the resulting spectra.

```

import numpy as np
import matplotlib.pyplot as plt
t = np.linspace(-5,5,1024)
dt = t[1]-t[0]
pulse1 = np.where(np.abs(t)<=0.5,1,0)
pulse2 = np.where(np.abs(t)<=2,1,0)
X1 = np.fft.fftshift(np.fft.fft(pulse1))
X2 = np.fft.fftshift(np.fft.fft(pulse2))
f = np.fft.fftshift(np.fft.fftfreq(len(t),dt))

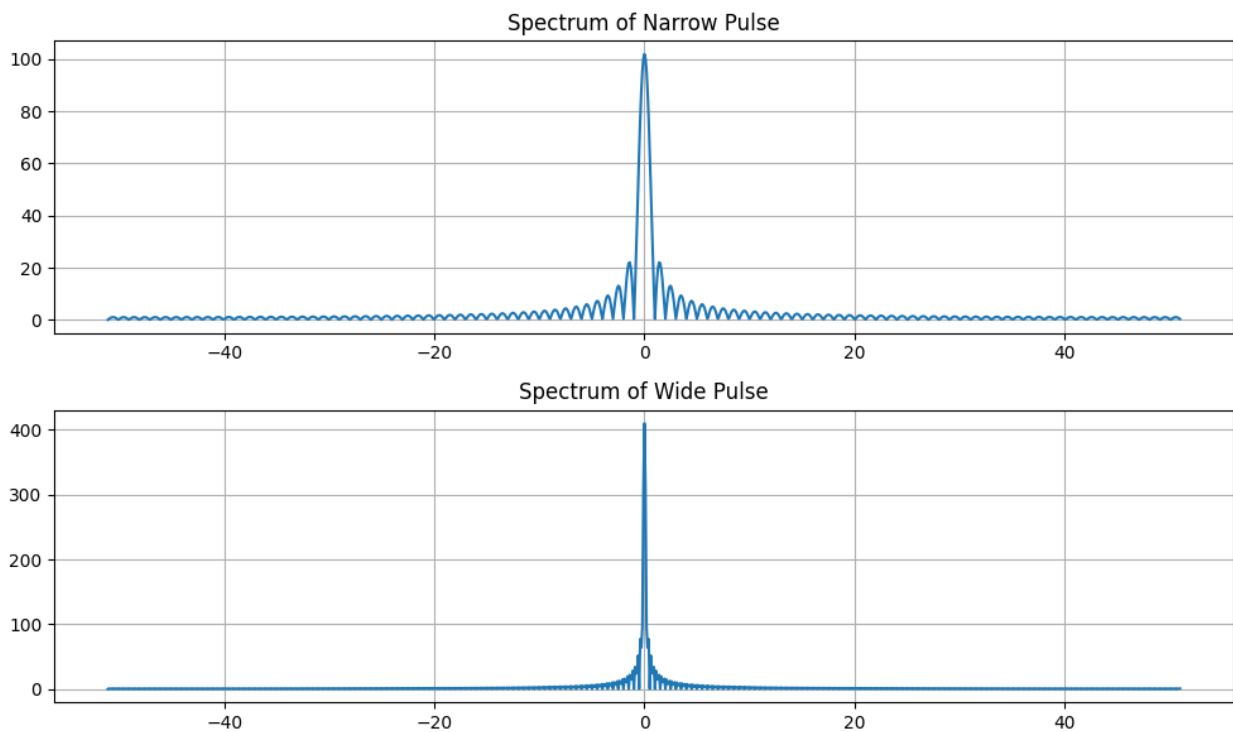
```

```
plt.figure(figsize=(10,6))

plt.subplot(2,1,1)
plt.plot(f,np.abs(X1))
plt.title("Spectrum of Narrow Pulse")
plt.grid(True)

plt.subplot(2,1,2)
plt.plot(f,np.abs(X2))
plt.title("Spectrum of Wide Pulse")
plt.grid(True)

plt.tight_layout()
plt.show()
```



Section B — Intermediate Problems

Problem 6

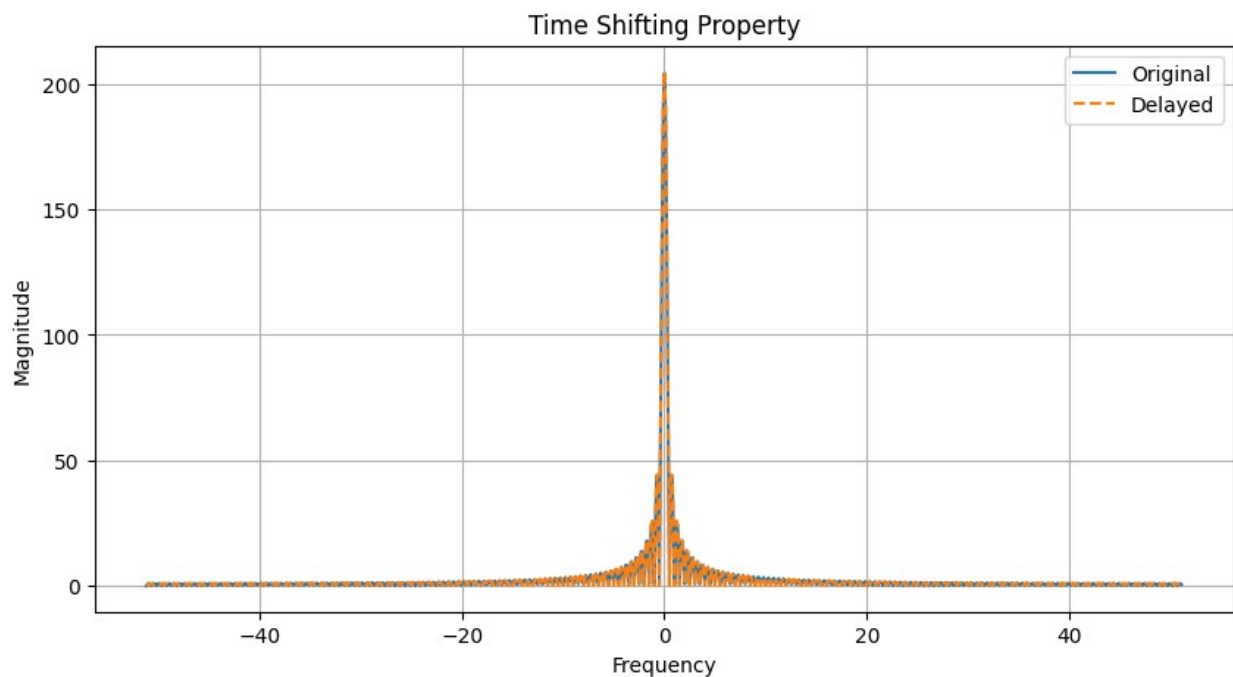
```
import numpy as np
import matplotlib.pyplot as plt
t = np.linspace(-5,5,1024) # Time vector
dt = t[1]-t[0]
x = np.where(np.abs(t)<=1,1,0) # Original signal
delay = 2
x_delay = np.where(np.abs(t-delay)<=1,1,0) # Delayed signal
X = np.fft.fftshift(np.fft.fft(x)) # FFT
```

```

X_delay = np.fft.fftshift(np.fft.fft(x_delay))
f = np.fft.fftshift(np.fft.fftfreq(len(t),dt))

plt.figure(figsize=(10,5))
plt.plot(f,np.abs(X),label="Original")
plt.plot(f,np.abs(X_delay),'--',label="Delayed")
plt.xlabel("Frequency")
plt.ylabel("Magnitude")
plt.title("Time Shifting Property")
plt.legend()
plt.grid()
plt.show()

```



Problem 7

Hint

Multiply a sinc signal by a cosine wave.

Compute the Fourier Transform.

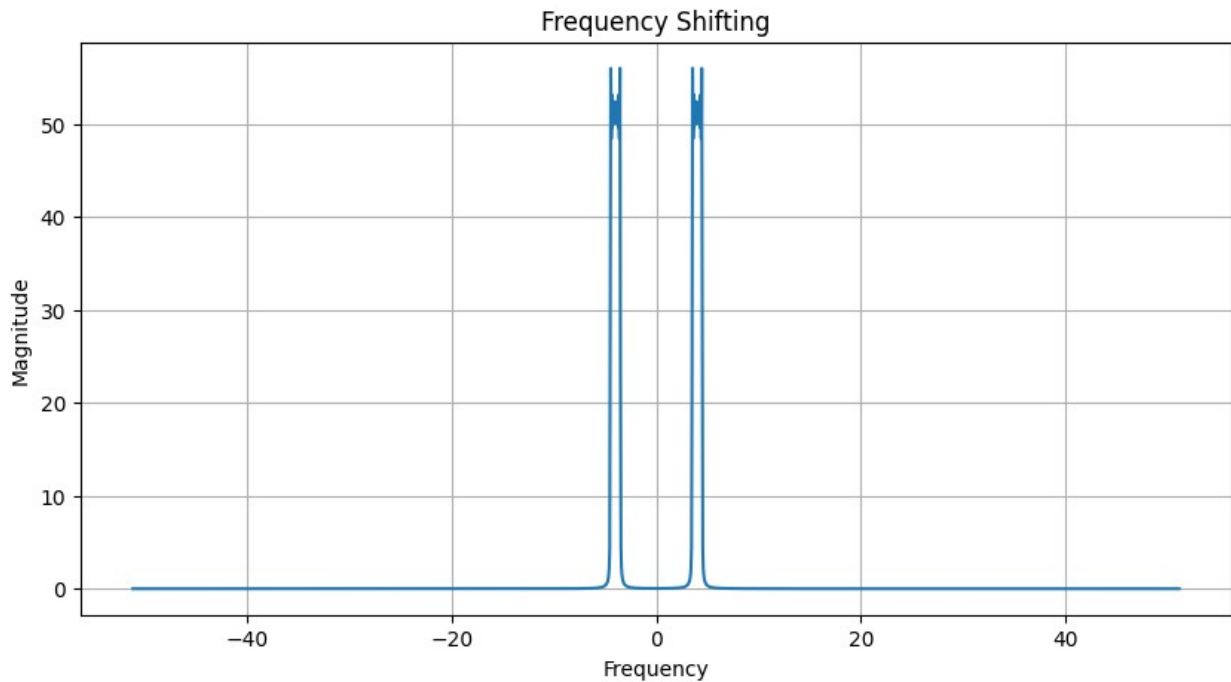
Observe how the spectrum shifts.

```

t = np.linspace(-10,10,2048)
dt = t[1]-t[0]
x = np.sinc(t)
carrier = np.cos(8*np.pi*t)
y = x*carrier
Y = np.fft.fftshift(np.fft.fft(y))

```

```
f = np.fft.fftshift(np.fft.fftfreq(len(t),dt))
plt.figure(figsize=(10,5))
plt.plot(f,np.abs(Y))
plt.title("Frequency Shifting")
plt.xlabel("Frequency")
plt.ylabel("Magnitude")
plt.grid()
plt.show()
```



Problem 8

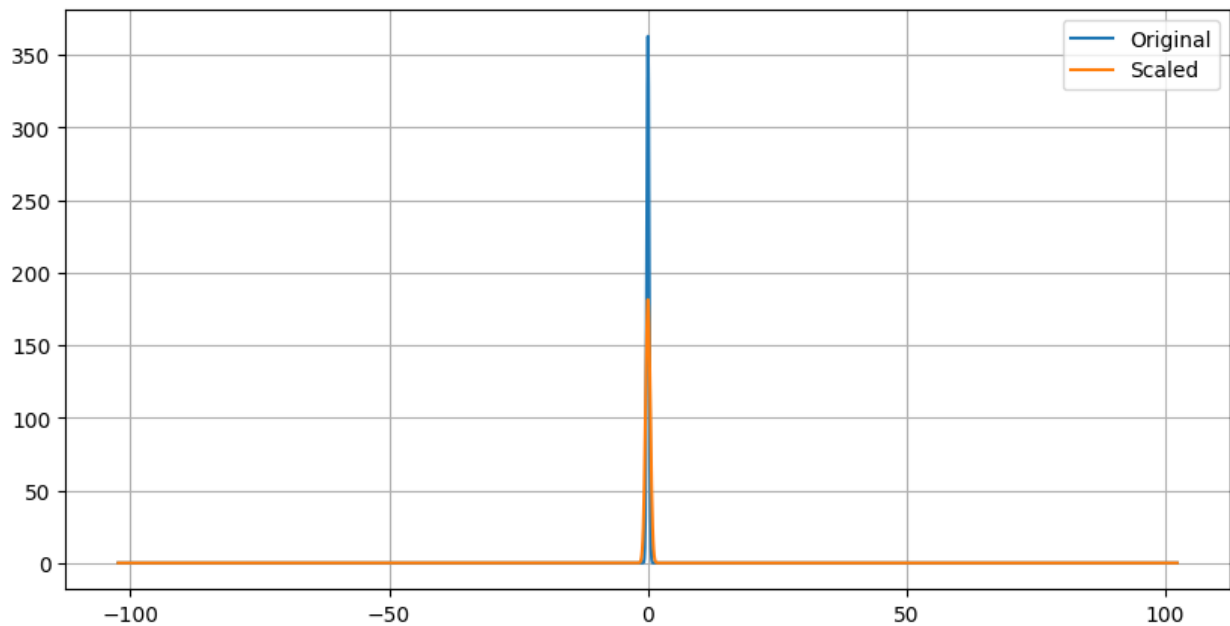
Hint

Generate $x(t)$ and $x(2t)$

Compare their Fourier spectra.

```
t = np.linspace(-5,5,2048)
dt = t[1]-t[0]
x1 = np.exp(-t**2)
x2 = np.exp(-(2*t)**2)
X1 = np.fft.fftshift(np.fft.fft(x1))
X2 = np.fft.fftshift(np.fft.fft(x2))
f = np.fft.fftshift(np.fft.fftfreq(len(t),dt))
plt.figure(figsize=(10,5))
plt.plot(f,np.abs(X1),label="Original")
plt.plot(f,np.abs(X2),label="Scaled")
plt.legend()
```

```
plt.grid()  
plt.show()
```

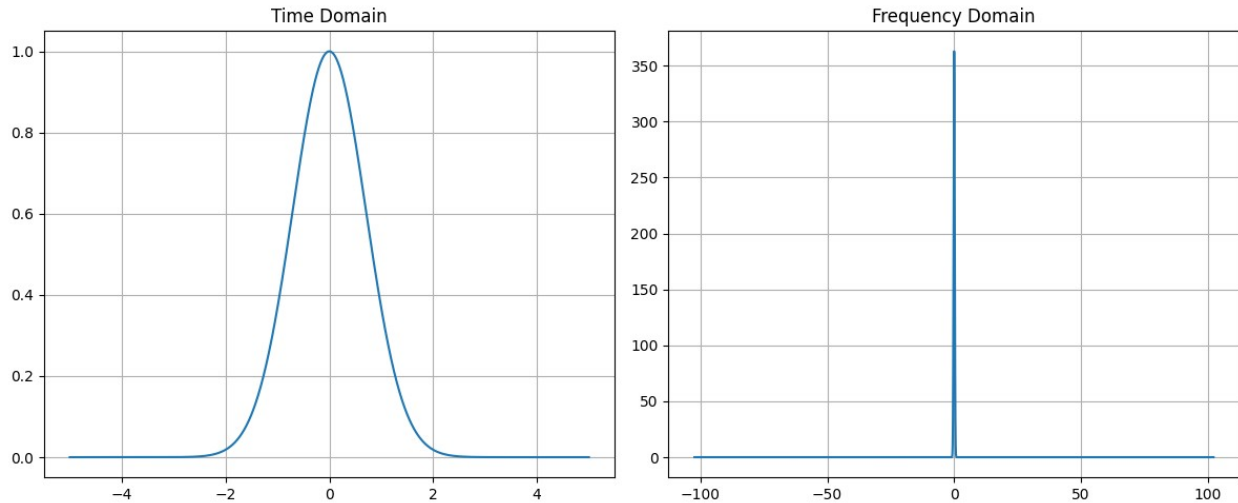


Problem 9

Hint

Generate a Gaussian signal. Observe similarities between the signal and its Fourier Transform.

```
t = np.linspace(-5,5,2048)  
dt = t[1]-t[0]  
x = np.exp(-t**2)  
X = np.fft.fftshift(np.fft.fft(x))  
f = np.fft.fftshift(np.fft.fftfreq(len(t),dt))  
plt.figure(figsize=(12,5))  
plt.subplot(1,2,1)  
plt.plot(t,x)  
plt.title("Time Domain")  
plt.grid()  
plt.subplot(1,2,2)  
plt.plot(f,np.abs(X))  
plt.title("Frequency Domain")  
plt.grid()  
plt.tight_layout()  
plt.show()
```

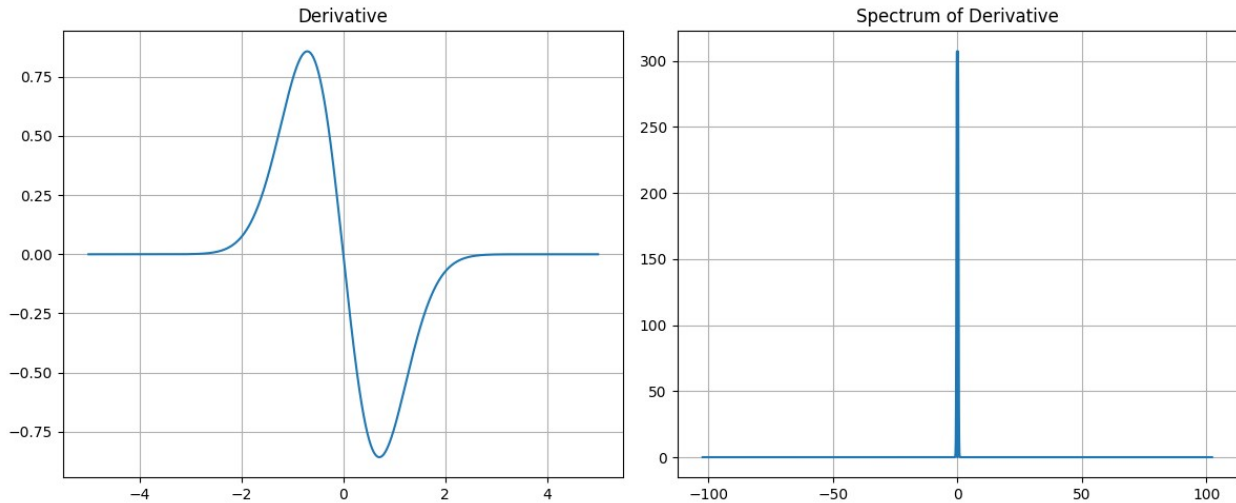


Problem 10

Hint

Differentiate a Gaussian signal numerically and compare its spectrum with the original signal.

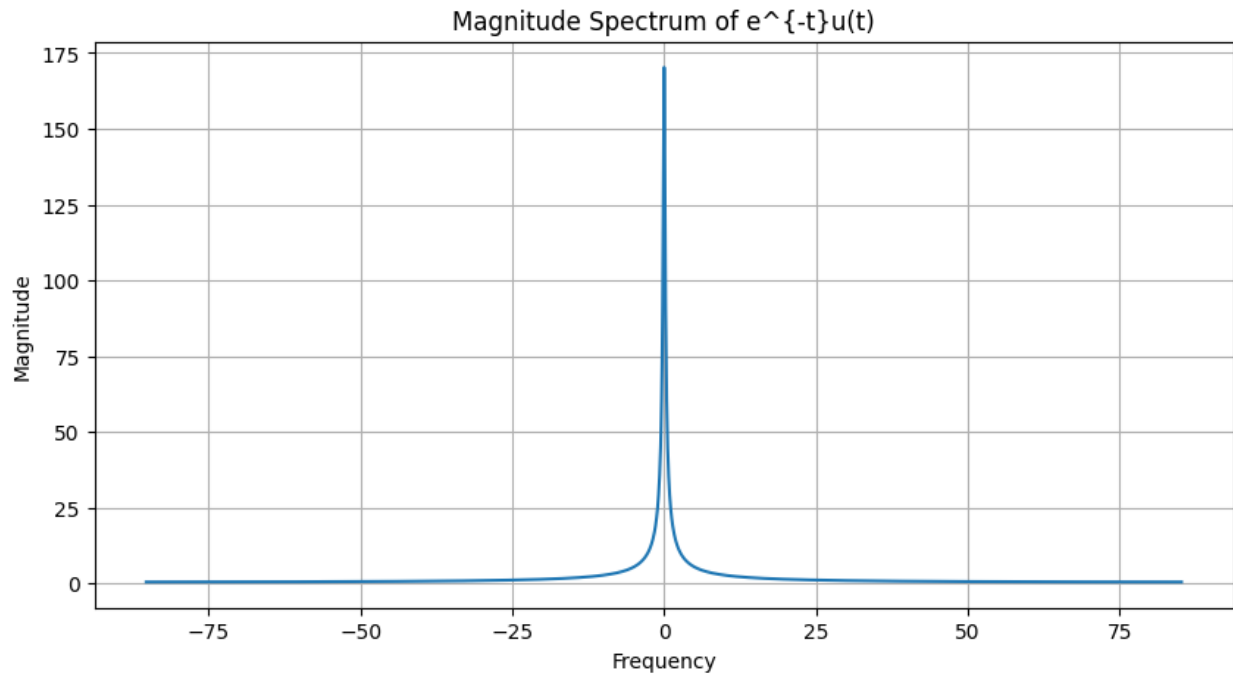
```
t = np.linspace(-5,5,2048)
dt = t[1]-t[0]
x = np.exp(-t**2)
dx = np.gradient(x,dt)
DX = np.fft.fftshift(np.fft.fft(dx))
f = np.fft.fftshift(np.fft.fftfreq(len(t),dt))
plt.figure(figsize=(12,5))
plt.subplot(1,2,1)
plt.plot(t,dx)
plt.title("Derivative")
plt.grid()
plt.subplot(1,2,2)
plt.plot(f,np.abs(DX))
plt.title("Spectrum of Derivative")
plt.grid()
plt.tight_layout()
plt.show()
```



Section C — Programming Problems

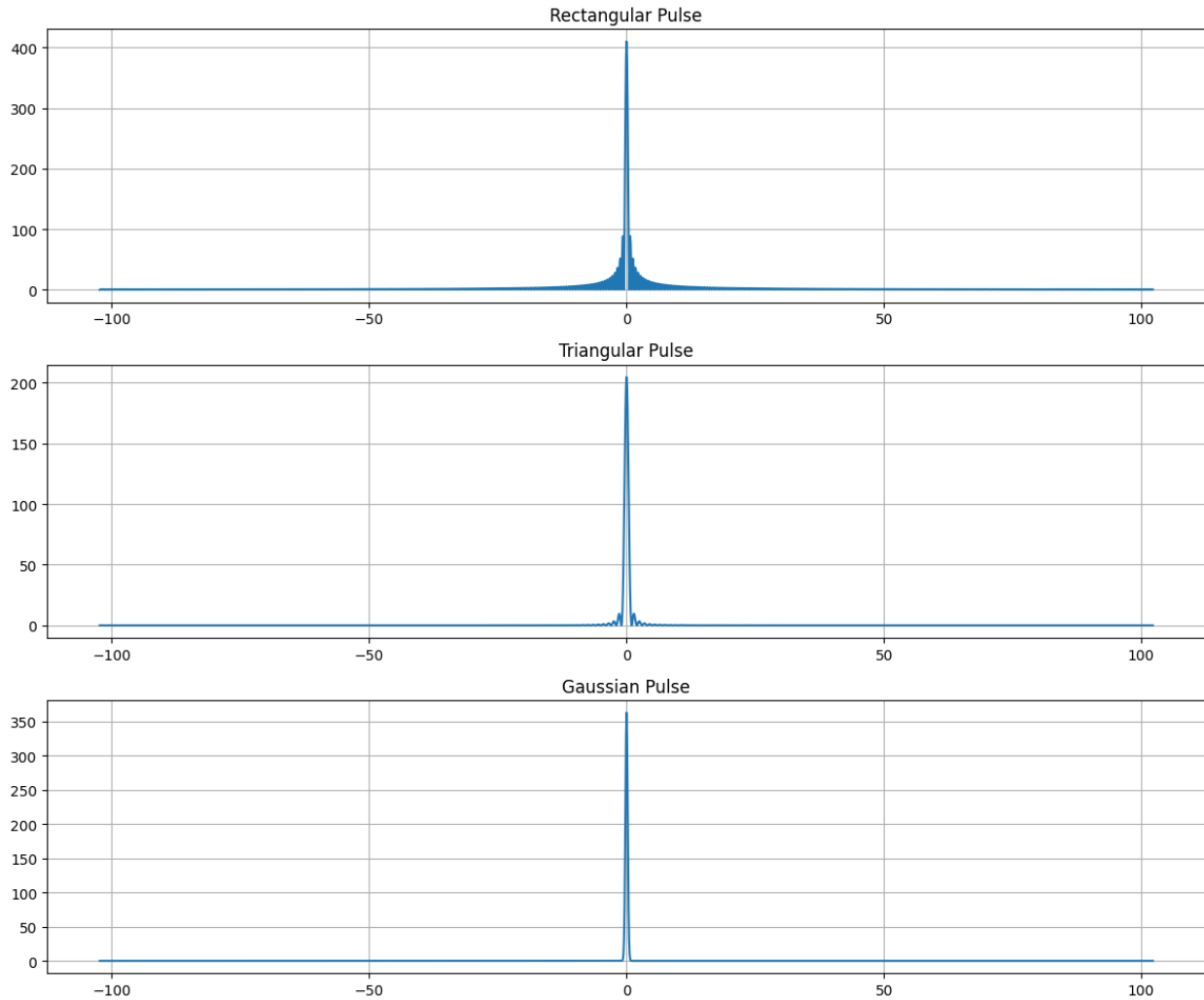
Problem 11

```
import numpy as np
import matplotlib.pyplot as plt
t = np.linspace(-2,10,2048) # Time axis
dt = t[1]-t[0]
u = np.where(t>=0,1,0) # Unit step
x = np.exp(-t)*u # Signal
X = np.fft.fftshift(np.fft.fft(x)) # Fourier Transform
f = np.fft.fftshift(np.fft.fftfreq(len(t),dt))
plt.figure(figsize=(10,5))
plt.plot(f,np.abs(X))
plt.title("Magnitude Spectrum of  $e^{-t}u(t)$ ")
plt.xlabel("Frequency")
plt.ylabel("Magnitude")
plt.grid(True)
plt.show()
```



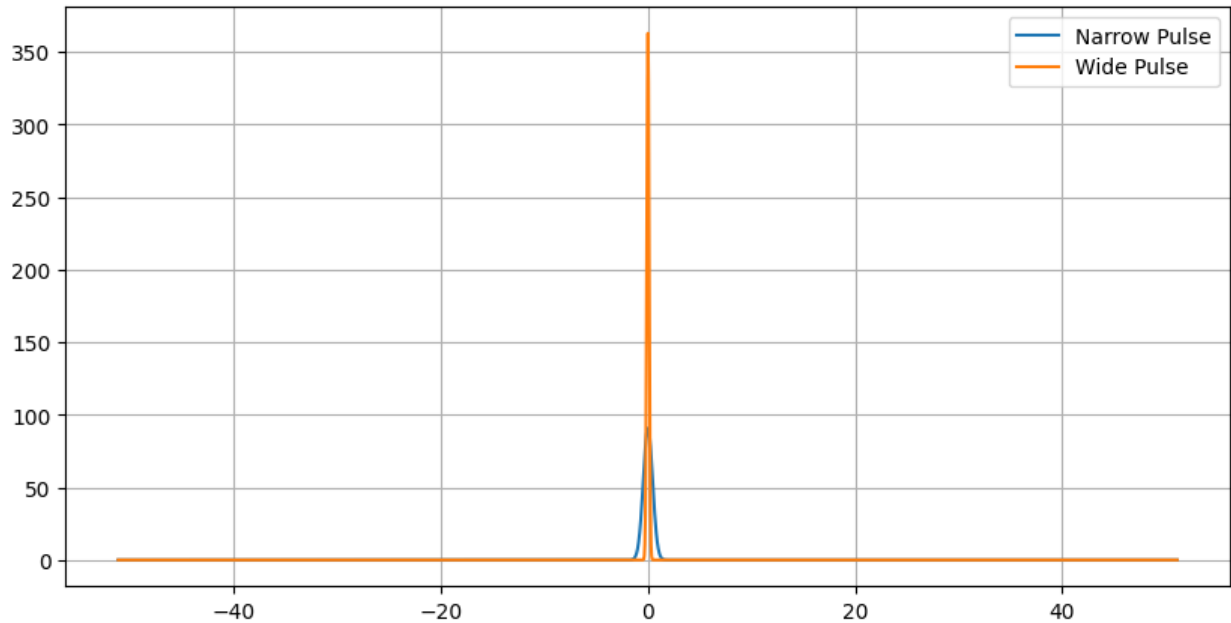
Problem 12

```
t = np.linspace(-5,5,2048)
dt = t[1]-t[0] if False else t[1]-t[0]
rect = np.where(np.abs(t)<=1,1,0)
tri = np.maximum(1-np.abs(t),0)
gauss = np.exp(-t**2)
signals = [rect,tri,gauss]
titles = ["Rectangular Pulse",
          "Triangular Pulse",
          "Gaussian Pulse"]
plt.figure(figsize=(12,10))
for i,s in enumerate(signals):
    X = np.fft.fftshift(np.fft.fft(s))
    f = np.fft.fftshift(np.fft.fftfreq(len(t),dt))
    plt.subplot(3,1,i+1)
    plt.plot(f,np.abs(X))
    plt.title(titles[i])
    plt.grid()
plt.tight_layout()
plt.show()
```



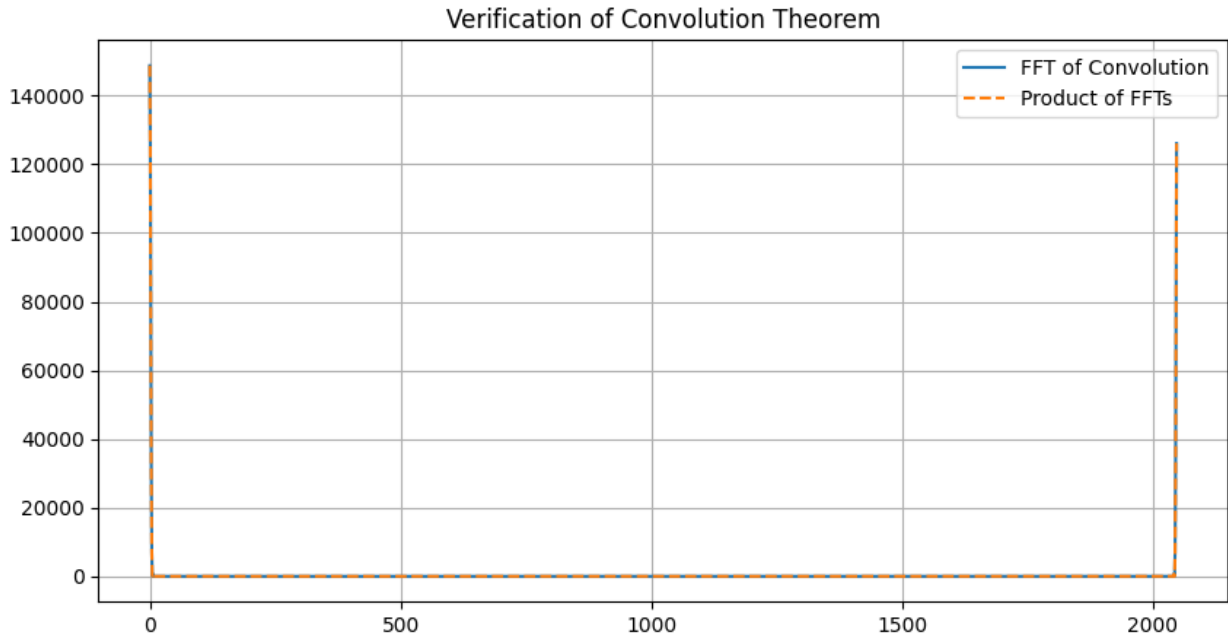
Problem 13

```
t = np.linspace(-10,10,2048)
dt = t[1]-t[0]
x1 = np.exp(-(t/0.5)**2)
x2 = np.exp(-(t/2)**2)
X1 = np.fft.fftshift(np.fft.fft(x1))
X2 = np.fft.fftshift(np.fft.fft(x2))
f = np.fft.fftshift(np.fft.fftfreq(len(t),dt))
plt.figure(figsize=(10,5))
plt.plot(f,np.abs(X1),label="Narrow Pulse")
plt.plot(f,np.abs(X2),label="Wide Pulse")
plt.legend()
plt.grid()
plt.show()
```



Problem 14

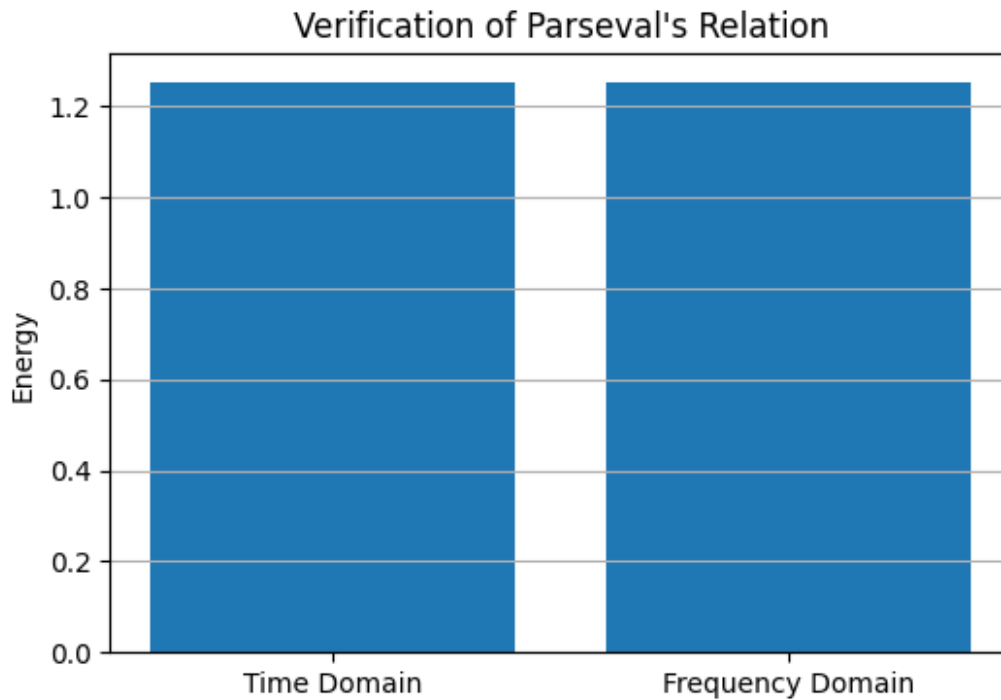
```
t = np.linspace(-5,5,2048)
dt = t[1]-t[0]
x = np.exp(-t**2)
h = np.where(np.abs(t)<=1,1,0)
y = np.convolve(x,h,mode='same')
X = np.fft.fft(x)
H = np.fft.fft(h)
Y = np.fft.fft(y)
plt.figure(figsize=(10,5))
plt.plot(np.abs(Y),label="FFT of Convolution")
plt.plot(np.abs(X*H),label="Product of FFTs",linestyle='--')
plt.legend()
plt.grid()
plt.title("Verification of Convolution Theorem")
plt.show()
```



Problem 15

```
import numpy as np
import matplotlib.pyplot as plt
# Time vector
t = np.linspace(-5, 5, 2048)
dt = t[1] - t[0]
# Signal
x = np.exp(-t**2)
# Time-domain energy
Et = np.trapezoid(np.abs(x)**2, t)
# FFT
X = np.fft.fftshift(np.fft.fft(x)) * dt
# Frequency axis
f = np.fft.fftshift(np.fft.fftfreq(len(t), d=dt))
df = f[1] - f[0]
# Frequency-domain energy
Ef = np.trapezoid(np.abs(X)**2, f)
print(f"Time Domain Energy      = {Et:.6f}")
print(f"Frequency Domain Energy = {Ef:.6f}")
# Comparison plot
plt.figure(figsize=(6,4))
plt.bar(["Time Domain", "Frequency Domain"], [Et, Ef])
plt.title("Verification of Parseval's Relation")
plt.ylabel("Energy")
plt.grid(axis="y")
plt.show()
```

Time Domain Energy = 1.253314
Frequency Domain Energy = 1.253314

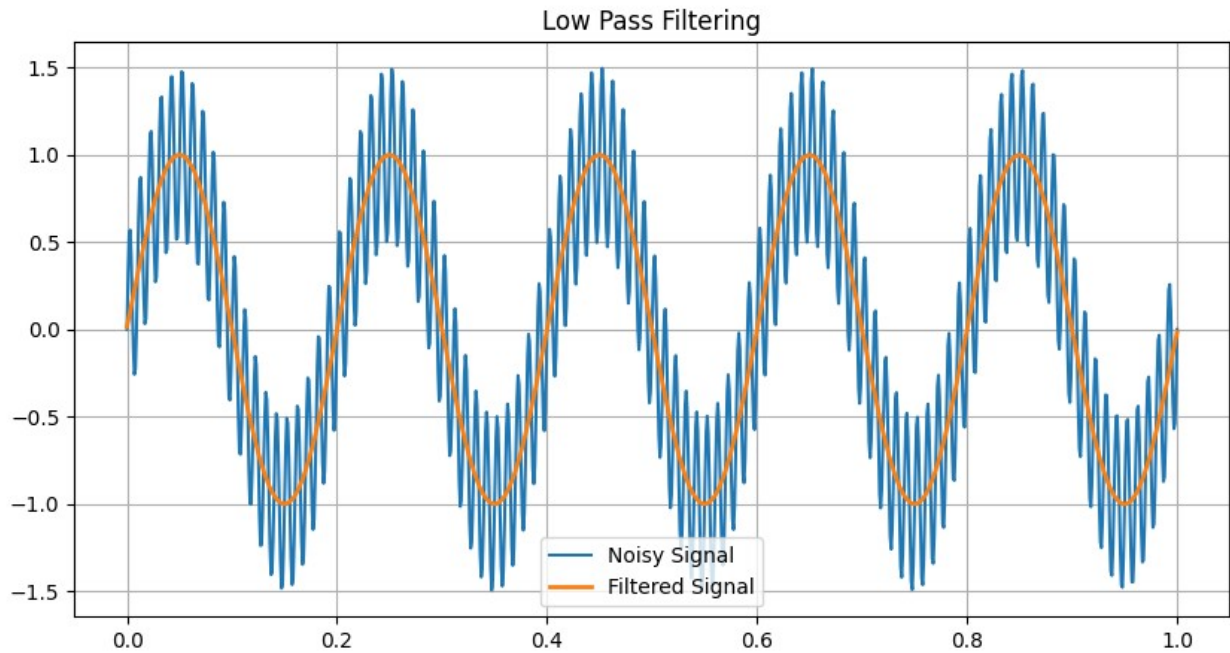


Challenge Problems

Challenge 1

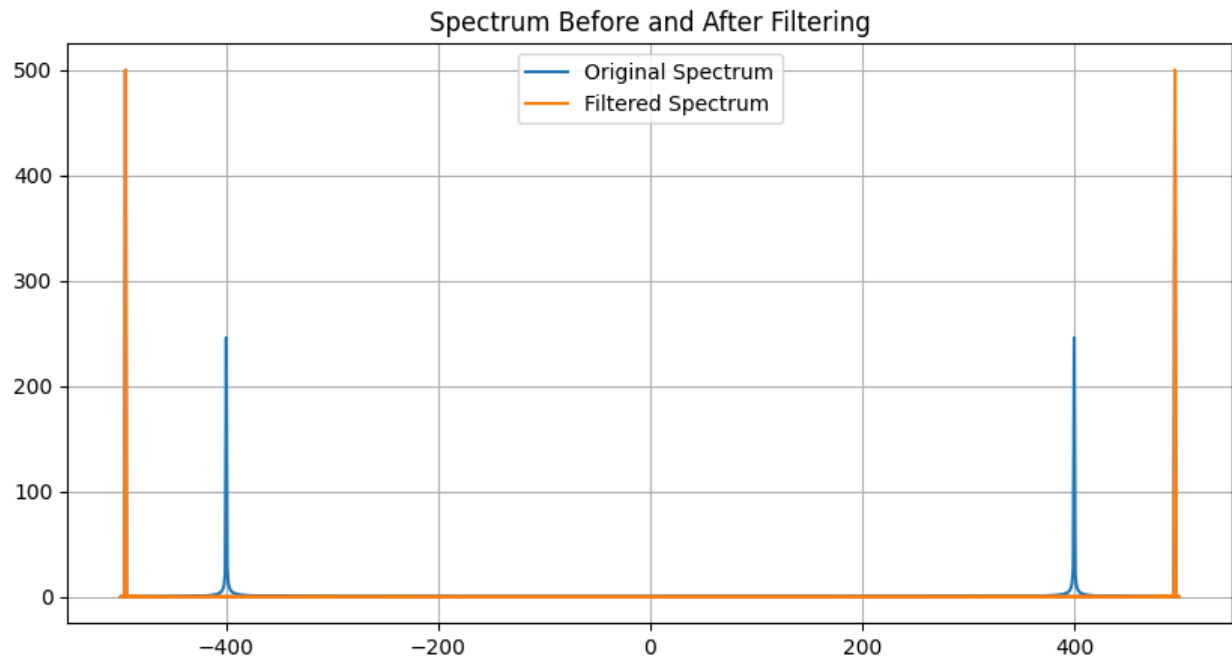
```
import numpy as np
import matplotlib.pyplot as plt
# Time axis
t = np.linspace(0,1,1000)
dt = t[1]-t[0]
# Original signal
signal = np.sin(2*np.pi*5*t)
# Add noise
noise = 0.5*np.sin(2*np.pi*100*t)
x = signal + noise
# FFT
X = np.fft.fft(x)
freq = np.fft.fftfreq(len(t),dt)
# Low-pass filter
cutoff = 20
X_filtered = X.copy()
X_filtered[np.abs(freq)>cutoff] = 0
# Inverse FFT
filtered = np.real(np.fft.ifft(X_filtered))
plt.figure(figsize=(10,5))
plt.plot(t,x,label="Noisy Signal")
plt.plot(t,filtered,label="Filtered Signal",linewidth=2)
plt.legend()
```

```
plt.grid()  
plt.title("Low Pass Filtering")  
plt.show()
```



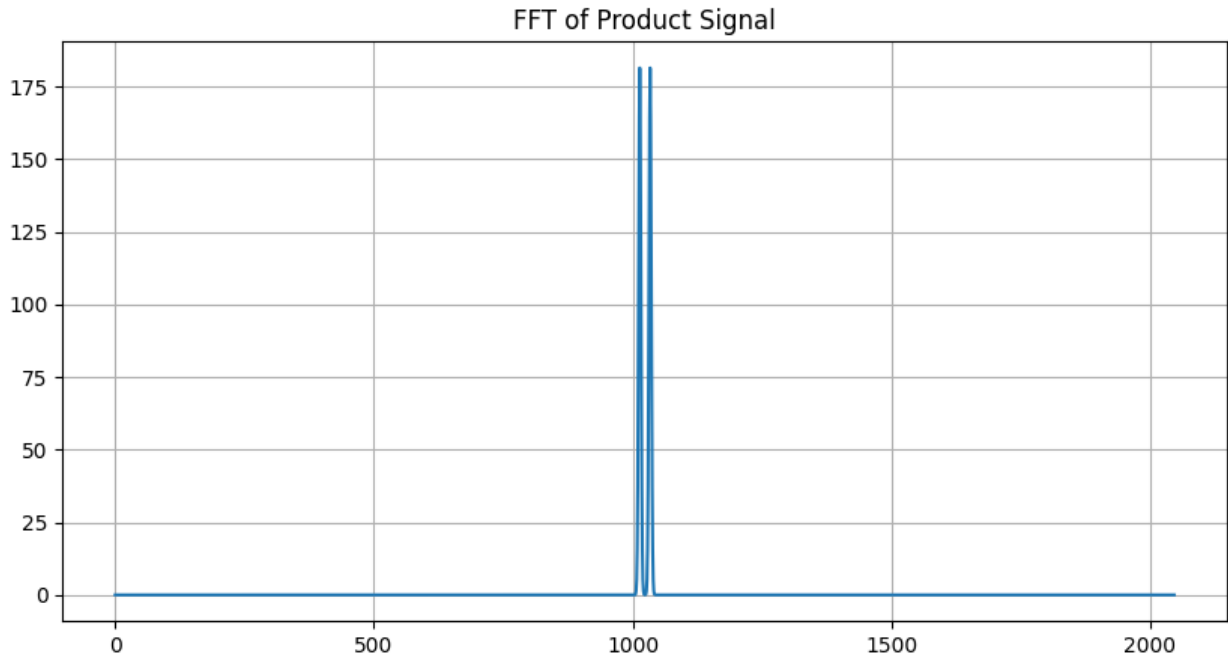
Challenge 2

```
plt.figure(figsize=(10,5))  
plt.plot(freq,np.abs(np.fft.fftshift(X)),label="Original Spectrum")  
plt.plot(freq,np.abs(np.fft.fftshift(X_filtered)),label="Filtered  
Spectrum")  
plt.legend()  
plt.grid()  
plt.title("Spectrum Before and After Filtering")  
plt.show()
```



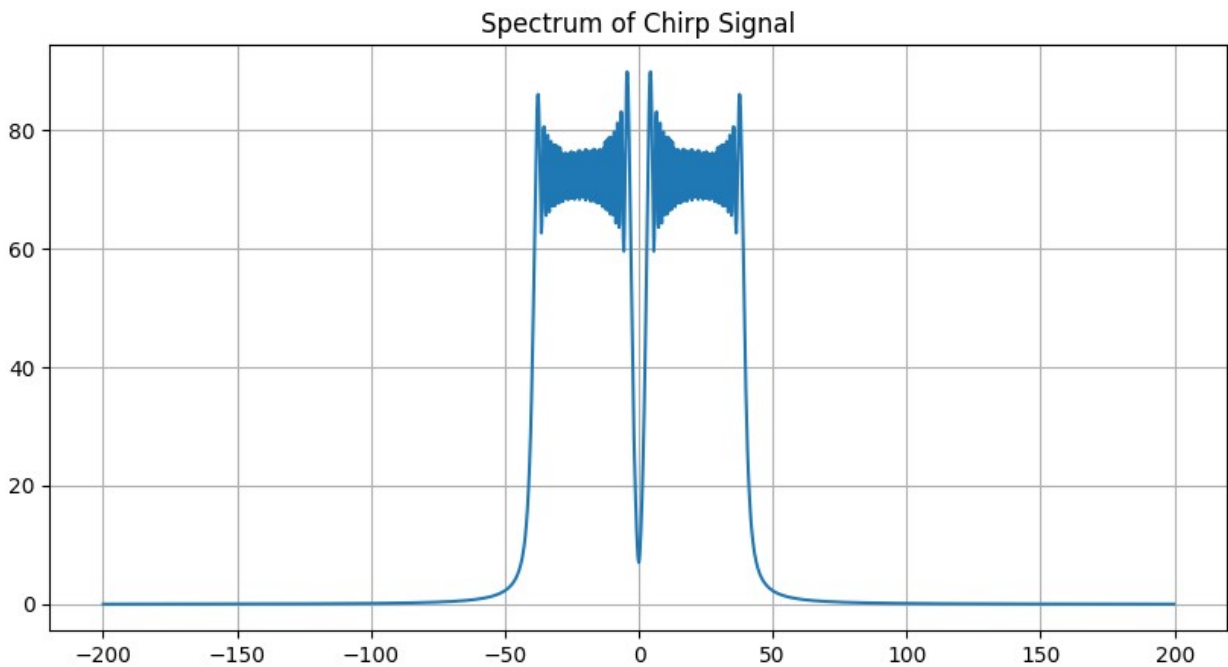
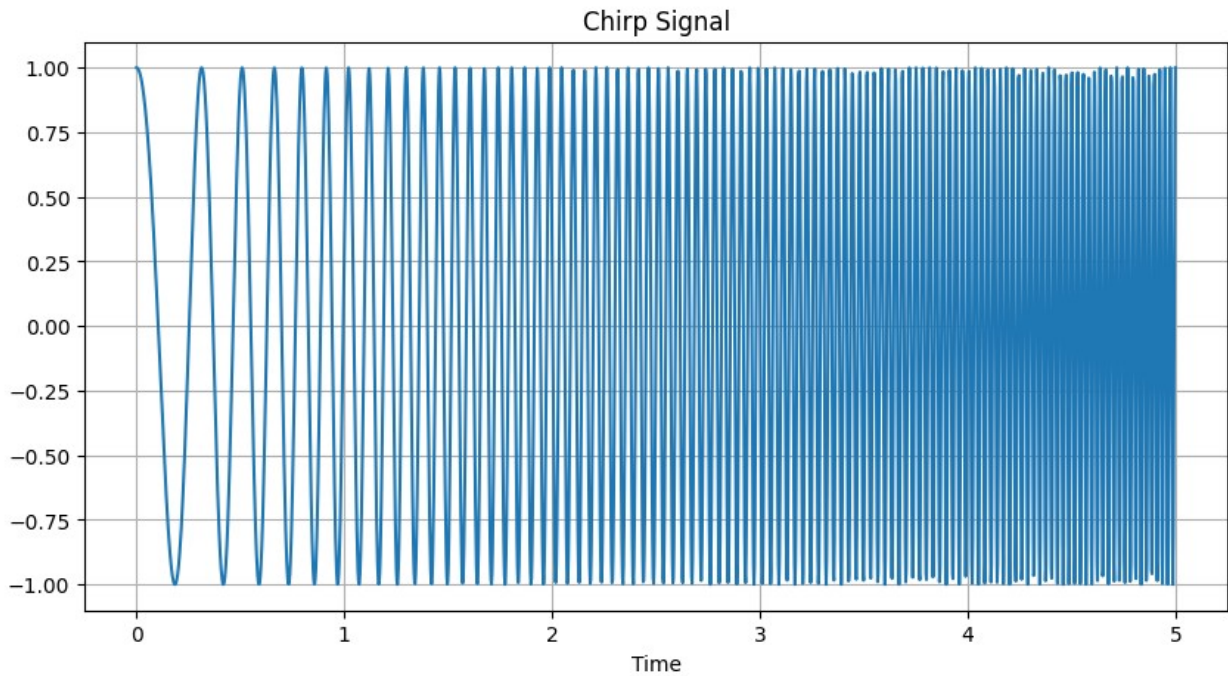
Challenge 3

```
t = np.linspace(-5,5,2048)
dt = t[1]-t[0]
x = np.cos(2*np.pi*t)
h = np.exp(-t**2)
y = x*h
X = np.fft.fftshift(np.fft.fft(x))
H = np.fft.fftshift(np.fft.fft(h))
Y = np.fft.fftshift(np.fft.fft(y))
plt.figure(figsize=(10,5))
plt.plot(np.abs(Y))
plt.title("FFT of Product Signal")
plt.grid()
plt.show()
```



Challenge 4

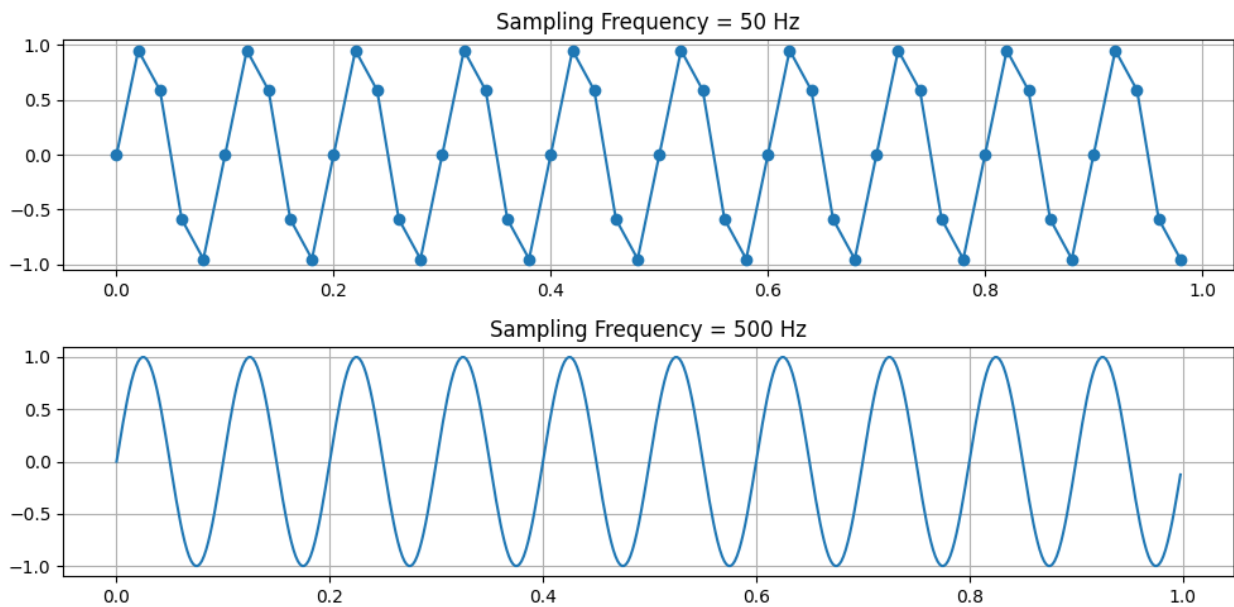
```
from scipy.signal import chirp
t = np.linspace(0,5,2000)
x = chirp(t,f0=2,f1=40,t1=5)
plt.figure(figsize=(10,5))
plt.plot(t,x)
plt.title("Chirp Signal")
plt.xlabel("Time")
plt.grid()
plt.show()
X = np.fft.fftshift(np.fft.fft(x))
f = np.fft.fftshift(np.fft.fftfreq(len(t),t[1]-t[0]))
plt.figure(figsize=(10,5))
plt.plot(f,np.abs(X))
plt.title("Spectrum of Chirp Signal")
plt.grid()
plt.show()
```



Challenge 5

```
fs1 = 50
fs2 = 500
t1 = np.arange(0,1,1/fs1)
t2 = np.arange(0,1,1/fs2)
x1 = np.sin(2*np.pi*10*t1)
x2 = np.sin(2*np.pi*10*t2)
```

```
plt.figure(figsize=(10,5))
plt.subplot(2,1,1)
plt.plot(t1,x1,'o-')
plt.title("Sampling Frequency = 50 Hz")
plt.grid()
plt.subplot(2,1,2)
plt.plot(t2,x2)
plt.title("Sampling Frequency = 500 Hz")
plt.grid()
plt.tight_layout()
plt.show()
```



Challenge 6

```
t = np.linspace(0,1,1000)
dt = t[1]-t[0]
x = np.sin(2*np.pi*10*t)+0.5*np.sin(2*np.pi*30*t)
X = np.fft.fftshift(np.fft.fft(x))
freq = np.fft.fftshift(np.fft.fftfreq(len(t),dt))
plt.figure(figsize=(12,8))
plt.subplot(3,1,1)
plt.plot(t,x)
plt.title("Time Domain Signal")
plt.grid()
plt.subplot(3,1,2)
plt.plot(freq,np.abs(X))
plt.title("Magnitude Spectrum")
plt.grid()
plt.subplot(3,1,3)
plt.plot(freq,np.angle(X))
```

```
plt.title("Phase Spectrum")  
plt.grid()  
plt.tight_layout()  
plt.show()
```

